

Notes on Chapter 19: Joins

Norman Lim

2025-08-04

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(nycflights13)
```

Important types of joins:

- **Mutating joins:** add new variables to one data frame from matching observations in another.
- **Filtering joins:** filter observations from one data frame based on whether or not they match an observation in another.

Keys

Primary and foreign keys

- **primary key:** a variable or set of variables that uniquely identifies each observation. When more than one variable is needed, the key is called a **compound key**.
- **foreign key:** a variable or set of variables that corresponds to a primary key in another table.

In this example, `carrier` is the primary key:

```
airlines
```

```
## # A tibble: 16 x 2
##   carrier name
##   <chr>    <chr>
## 1 9E      Endeavor Air Inc.
## 2 AA      American Airlines Inc.
## 3 AS      Alaska Airlines Inc.
## 4 B6      JetBlue Airways
## 5 DL      Delta Air Lines Inc.
## 6 EV      ExpressJet Airlines Inc.
## 7 F9      Frontier Airlines Inc.
## 8 FL      AirTran Airways Corporation
```

```
## 9 HA Hawaiian Airlines Inc.
## 10 MQ Envoy Air
## 11 OO SkyWest Airlines Inc.
## 12 UA United Air Lines Inc.
## 13 US US Airways Inc.
## 14 VX Virgin America
## 15 WN Southwest Airlines Co.
## 16 YV Mesa Airlines Inc.
```

Here, faa is the primary key:

airports

```
## # A tibble: 1,458 x 8
##   faa   name                lat   lon   alt   tz dst   tzone
##   <chr> <chr>                <dbl> <dbl> <dbl> <dbl> <chr> <chr>
## 1 04G   Lansdowne Airport      41.1  -80.6  1044   -5 A   America/~
## 2 06A   Moton Field Municipal Airport 32.5  -85.7   264   -6 A   America/~
## 3 06C   Schaumburg Regional    42.0  -88.1   801   -6 A   America/~
## 4 06N   Randall Airport        41.4  -74.4   523   -5 A   America/~
## 5 09J   Jekyll Island Airport   31.1  -81.4    11   -5 A   America/~
## 6 0A9   Elizabethton Municipal Airport 36.4  -82.2  1593   -5 A   America/~
## 7 0G6   Williams County Airport 41.5  -84.5   730   -5 A   America/~
## 8 0G7   Finger Lakes Regional Airport 42.9  -76.8   492   -5 A   America/~
## 9 0P2   Shoestring Aviation Airfield 39.8  -76.6  1000   -5 U   America/~
## 10 0S9   Jefferson County Intl    48.1 -123.   108   -8 A   America/~
## # i 1,448 more rows
```

Here, tailnum is the primary key:

planes

```
## # A tibble: 3,322 x 9
##   tailnum year type                manufacturer model engines seats speed engine
##   <chr>   <int> <chr>                <chr>         <chr>   <int> <int> <int> <chr>
## 1 N10156 2004 Fixed wing multi~ EMBRAER      EMB~         2    55   NA Turbo~
## 2 N102UW 1998 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## 3 N103US 1999 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## 4 N104UW 1999 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## 5 N10575 2002 Fixed wing multi~ EMBRAER      EMB~         2    55   NA Turbo~
## 6 N105UW 1999 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## 7 N107US 1999 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## 8 N108UW 1999 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## 9 N109UW 1999 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## 10 N110UW 1999 Fixed wing multi~ AIRBUS INDU~ A320~         2   182   NA Turbo~
## # i 3,312 more rows
```

Here, origin and time_hour are the compound key:

weather

```
## # A tibble: 26,115 x 15
##   origin year month   day hour  temp  dewp humid wind_dir wind_speed
##   <chr>   <int> <int> <int> <int> <dbl> <dbl> <dbl>   <dbl>   <dbl>
## 1 EWR    2013     1     1     1  39.0  26.1  59.4    270    10.4
## 2 EWR    2013     1     1     2  39.0  27.0  61.6    250     8.06
## 3 EWR    2013     1     1     3  39.0  28.0  64.4    240    11.5
## 4 EWR    2013     1     1     4  39.9  28.0  62.2    250    12.7
```

```
## 5 EWR      2013      1      1      5 39.0 28.0 64.4      260      12.7
## 6 EWR      2013      1      1      6 37.9 28.0 67.2      240      11.5
## 7 EWR      2013      1      1      7 39.0 28.0 64.4      240      15.0
## 8 EWR      2013      1      1      8 39.9 28.0 62.2      250      10.4
## 9 EWR      2013      1      1      9 39.9 28.0 62.2      260      15.0
## 10 EWR     2013      1      1     10 41    28.0 59.6      260      13.8
## # i 26,105 more rows
## # i 5 more variables: wind_gust <dbl>, precip <dbl>, pressure <dbl>,
## #   visib <dbl>, time_hour <dtm>
```

- In `flights`, the foreign key is `tailnum` which corresponds to the primary key in `planes`.
- In `flights`, `carrier` is another foreign key that corresponds to the primary key `carrier` in the `airlines` data. In `flights`, `flights$origin-flights$time_hour` is a compound foreign key that corresponds to the compound primary key `weather$origin-weather$time_hour`.

Checking primary keys

- Using `count()` to check if the variable is a unique identifier:

```
planes |>
  count(tailnum) |>
  filter(n > 1)
```

```
## # A tibble: 0 x 2
## # i 2 variables: tailnum <chr>, n <int>
```

```
weather |>
  count(time_hour, origin) |>
  filter(n > 1)
```

```
## # A tibble: 0 x 3
## # i 3 variables: time_hour <dtm>, origin <chr>, n <int>
```

- It is also a good idea to check for missing values in your primary keys – if a value is missing, then it can't identify an observation:

```
planes |>
  filter(is.na(tailnum))
```

```
## # A tibble: 0 x 9
## # i 9 variables: tailnum <chr>, year <int>, type <chr>, manufacturer <chr>,
## #   model <chr>, engines <int>, seats <int>, speed <int>, engine <chr>
```

```
weather |>
  filter(is.na(time_hour) | is.na(origin))
```

```
## # A tibble: 0 x 15
## # i 15 variables: origin <chr>, year <int>, month <int>, day <int>, hour <int>,
## #   temp <dbl>, dewp <dbl>, humid <dbl>, wind_dir <dbl>, wind_speed <dbl>,
## #   wind_gust <dbl>, precip <dbl>, pressure <dbl>, visib <dbl>,
## #   time_hour <dtm>
```

Surrogate keys

After a little thinking and experimentation, we determined that there are three variables that together uniquely identify each flight:

```
flights |>
  count(time_hour, carrier, flight) |>
  filter(n > 1)
```

```
## # A tibble: 0 x 4
## # i 4 variables: time_hour <dtm>, carrier <chr>, flight <int>, n <int>
```

Does the absence of duplicates automatically make `time_hour-carrier-flight` a primary key? Well, not exactly. But in this case, it seems reasonable because it would really be confusing for the passengers and the airline itself if there were multiple flights with the same flight number in the air at the same time.

Look at `alt` and `lat` in `airports`:

```
airports |>
  count(alt, lat) |>
  filter(n > 1)
```

```
## # A tibble: 1 x 3
##   alt   lat   n
##   <dbl> <dbl> <int>
## 1    13  40.6     2
```

It's not reasonable to use altitude and latitude to identify an airport.

A better solution is to create a numeric surrogate key:

```
flights2 <- flights |>
  mutate(id = row_number(), .before = 1)
```

```
flights2
```

```
## # A tibble: 336,776 x 20
##   id year month day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int> <int>   <int>         <int>         <dbl>   <int>
## 1     1  2013     1     1     517           515           2     830
## 2     2  2013     1     1     533           529           4     850
## 3     3  2013     1     1     542           540           2     923
## 4     4  2013     1     1     544           545          -1    1004
## 5     5  2013     1     1     554           600          -6     812
## 6     6  2013     1     1     554           558          -4     740
## 7     7  2013     1     1     555           600          -5     913
## 8     8  2013     1     1     557           600          -3     709
## 9     9  2013     1     1     557           600          -3     838
## 10    10  2013     1     1     558           600          -2     753
## # i 336,766 more rows
## # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>, carrier <chr>,
## #   flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
## #   distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
```

Exercises

1. We forgot to draw the relationship between `weather` and `airports`. What is the relationship and how should it appear in the diagram?
2. `weather` only contains information for the three origin airports in NYC. If it contained weather records for all airports in the USA, what additional connection would it make to `flights`?

Ans. State.

3. The `year`, `month`, `day`, `hour`, and `origin` variables almost form a compound key for `weather`, but there's one hour that has duplicate observations. Can you figure out what's special about that hour?
4. We know that some days of the year are special and fewer people than fly on them (e.g. Christmas eve and Christmas day). How might you represent that data as a data frame? What would be the primary key? How would it connect to the existing data frames?
5. Draw a diagram illustrating the connections between the `Batting`, `People`, and `Salaries` data frames in the `Lahman` package. Draw another diagram that shows the relationship between `People`, `Managers`, `AwardsManagers`. How would you characterize the relationship between the `Batting`, `Pitching`, and `Fielding` data frames?

Basic joins

Mutating joins

- **mutating join** allows users to combine variables from two data frames.

Examples:

```
# Create the base data frame
flights2 <- flights |>
  select(year, time_hour, origin, dest, tailnum, carrier)
```

flights2

```
## # A tibble: 336,776 x 6
##   year time_hour      origin dest  tailnum carrier
##   <int> <dtm>      <chr>  <chr> <chr>   <chr>
## 1  2013 2013-01-01 05:00:00 EWR   IAH   N14228 UA
## 2  2013 2013-01-01 05:00:00 LGA   IAH   N24211 UA
## 3  2013 2013-01-01 05:00:00 JFK   MIA   N619AA AA
## 4  2013 2013-01-01 05:00:00 JFK   BQN   N804JB B6
## 5  2013 2013-01-01 06:00:00 LGA   ATL   N668DN DL
## 6  2013 2013-01-01 05:00:00 EWR   ORD   N39463 UA
## 7  2013 2013-01-01 06:00:00 EWR   FLL   N516JB B6
## 8  2013 2013-01-01 06:00:00 LGA   IAD   N829AS EV
## 9  2013 2013-01-01 06:00:00 JFK   MCO   N593JB B6
## 10 2013 2013-01-01 06:00:00 LGA   ORD   N3ALAA AA
## # i 336,766 more rows
```

- Using left-join to add more metadata but retain the number of rows

```
flights2 |>
  left_join(airlines)
```

```
## Joining with `by = join_by(carrier)`
```

```
## # A tibble: 336,776 x 7
##   year time_hour      origin dest  tailnum carrier name
##   <int> <dtm>      <chr>  <chr> <chr>   <chr>  <chr>
## 1  2013 2013-01-01 05:00:00 EWR   IAH   N14228 UA      United Air Lines Inc.
## 2  2013 2013-01-01 05:00:00 LGA   IAH   N24211 UA      United Air Lines Inc.
## 3  2013 2013-01-01 05:00:00 JFK   MIA   N619AA AA      American Airlines Inc.
## 4  2013 2013-01-01 05:00:00 JFK   BQN   N804JB B6      JetBlue Airways
## 5  2013 2013-01-01 06:00:00 LGA   ATL   N668DN DL      Delta Air Lines Inc.
## 6  2013 2013-01-01 05:00:00 EWR   ORD   N39463 UA      United Air Lines Inc.
## 7  2013 2013-01-01 06:00:00 EWR   FLL   N516JB B6      JetBlue Airways
## 8  2013 2013-01-01 06:00:00 LGA   IAD   N829AS EV      ExpressJet Airlines I~
```

```
## 9 2013 2013-01-01 06:00:00 JFK MCO N593JB B6 JetBlue Airways
## 10 2013 2013-01-01 06:00:00 LGA ORD N3ALAA AA American Airlines Inc.
## # i 336,766 more rows
```

- Or to add temperature and wind speed when each plane departed:

```
flights2 |>
  left_join(weather |> select(origin, time_hour, temp, wind_speed))
```

```
## Joining with `by = join_by(time_hour, origin)`
```

```
## # A tibble: 336,776 x 8
```

```
##   year time_hour      origin dest tailnum carrier temp wind_speed
##   <int> <dtm>      <chr>  <chr> <chr>  <chr>  <dbl>   <dbl>
## 1 2013 2013-01-01 05:00:00 EWR   IAH  N14228  UA    39.0    12.7
## 2 2013 2013-01-01 05:00:00 LGA   IAH  N24211  UA    39.9    15.0
## 3 2013 2013-01-01 05:00:00 JFK   MIA  N619AA  AA    39.0    15.0
## 4 2013 2013-01-01 05:00:00 JFK   BQN  N804JB  B6    39.0    15.0
## 5 2013 2013-01-01 06:00:00 LGA   ATL  N668DN  DL    39.9    16.1
## 6 2013 2013-01-01 05:00:00 EWR   ORD  N39463  UA    39.0    12.7
## 7 2013 2013-01-01 06:00:00 EWR   FLL  N516JB  B6    37.9    11.5
## 8 2013 2013-01-01 06:00:00 LGA   IAD  N829AS  EV    39.9    16.1
## 9 2013 2013-01-01 06:00:00 JFK   MCO  N593JB  B6    37.9    13.8
## 10 2013 2013-01-01 06:00:00 LGA   ORD  N3ALAA  AA    39.9    16.1
## # i 336,766 more rows
```

- Or to determine what size of plane was flying:

```
flights2 |>
  left_join(planes |> select(tailnum, type, engines, seats))
```

```
## Joining with `by = join_by(tailnum)`
```

```
## # A tibble: 336,776 x 9
```

```
##   year time_hour      origin dest tailnum carrier type engines seats
##   <int> <dtm>      <chr>  <chr> <chr>  <chr>  <chr>   <int> <int>
## 1 2013 2013-01-01 05:00:00 EWR   IAH  N14228  UA    Fixed w~      2    149
## 2 2013 2013-01-01 05:00:00 LGA   IAH  N24211  UA    Fixed w~      2    149
## 3 2013 2013-01-01 05:00:00 JFK   MIA  N619AA  AA    Fixed w~      2    178
## 4 2013 2013-01-01 05:00:00 JFK   BQN  N804JB  B6    Fixed w~      2    200
## 5 2013 2013-01-01 06:00:00 LGA   ATL  N668DN  DL    Fixed w~      2    178
## 6 2013 2013-01-01 05:00:00 EWR   ORD  N39463  UA    Fixed w~      2    191
## 7 2013 2013-01-01 06:00:00 EWR   FLL  N516JB  B6    Fixed w~      2    200
## 8 2013 2013-01-01 06:00:00 LGA   IAD  N829AS  EV    Fixed w~      2     55
## 9 2013 2013-01-01 06:00:00 JFK   MCO  N593JB  B6    Fixed w~      2    200
## 10 2013 2013-01-01 06:00:00 LGA   ORD  N3ALAA  AA    <NA>         NA     NA
## # i 336,766 more rows
```

- When `left_join` fails to find a match for a row in `x`, it fills in the new variables with missing values.

```
flights2 |>
  filter(tailnum == "N3ALAA") |>
  left_join(planes |> select(tailnum, type, engines, seats))
```

```
## Joining with `by = join_by(tailnum)`
```

```
## # A tibble: 63 x 9
```

```
##   year time_hour      origin dest tailnum carrier type engines seats
##   <int> <dtm>      <chr>  <chr> <chr>  <chr>  <chr>   <int> <int>
```

```
## 1 2013 2013-01-01 06:00:00 LGA ORD N3ALAA AA <NA> NA NA
## 2 2013 2013-01-02 18:00:00 LGA ORD N3ALAA AA <NA> NA NA
## 3 2013 2013-01-03 06:00:00 LGA ORD N3ALAA AA <NA> NA NA
## 4 2013 2013-01-07 19:00:00 LGA ORD N3ALAA AA <NA> NA NA
## 5 2013 2013-01-08 17:00:00 JFK ORD N3ALAA AA <NA> NA NA
## 6 2013 2013-01-16 06:00:00 LGA ORD N3ALAA AA <NA> NA NA
## 7 2013 2013-01-20 18:00:00 LGA ORD N3ALAA AA <NA> NA NA
## 8 2013 2013-01-22 17:00:00 JFK ORD N3ALAA AA <NA> NA NA
## 9 2013 2013-10-11 06:00:00 EWR MIA N3ALAA AA <NA> NA NA
## 10 2013 2013-10-14 08:00:00 JFK BOS N3ALAA AA <NA> NA NA
## # i 53 more rows
```

Specifying join keys

- The default action of `left_join()` is to use all variables that appear in both data frames as the join key (aka natural join). But this doesn't always work. Consider the example below:

```
flights2 |>
  left_join(planes)
```

```
## Joining with `by = join_by(year, tailnum)`
## # A tibble: 336,776 x 13
##   year time_hour origin dest tailnum carrier type manufacturer
##   <int> <dtm>    <chr> <chr> <chr> <chr> <chr> <chr>
## 1 2013 2013-01-01 05:00:00 EWR IAH N14228 UA <NA> <NA>
## 2 2013 2013-01-01 05:00:00 LGA IAH N24211 UA <NA> <NA>
## 3 2013 2013-01-01 05:00:00 JFK MIA N619AA AA <NA> <NA>
## 4 2013 2013-01-01 05:00:00 JFK BQN N804JB B6 <NA> <NA>
## 5 2013 2013-01-01 06:00:00 LGA ATL N668DN DL <NA> <NA>
## 6 2013 2013-01-01 05:00:00 EWR ORD N39463 UA <NA> <NA>
## 7 2013 2013-01-01 06:00:00 EWR FLL N516JB B6 <NA> <NA>
## 8 2013 2013-01-01 06:00:00 LGA IAD N829AS EV <NA> <NA>
## 9 2013 2013-01-01 06:00:00 JFK MCO N593JB B6 <NA> <NA>
## 10 2013 2013-01-01 06:00:00 LGA ORD N3ALAA AA <NA> <NA>
## # i 336,766 more rows
## # i 5 more variables: model <chr>, engines <int>, seats <int>, speed <int>,
## #   engine <chr>
```

The result contains a low of missing matches because `left_join` tried to use `tailnum` and `year` as a compound key, but in the `flights` and `planes` datasets, 'year' has a different meaning: year of flight date for the former, and year when the plane was built, for the latter.

Using `left_join()` with the joining key specified explicitly using the `join_by` argument:

```
flights2 |>
  left_join(planes, join_by(tailnum))
```

```
## # A tibble: 336,776 x 14
##   year.x time_hour origin dest tailnum carrier year.y type
##   <int> <dtm>    <chr> <chr> <chr> <chr> <int> <chr>
## 1 2013 2013-01-01 05:00:00 EWR IAH N14228 UA 1999 Fixed wing mu~
## 2 2013 2013-01-01 05:00:00 LGA IAH N24211 UA 1998 Fixed wing mu~
## 3 2013 2013-01-01 05:00:00 JFK MIA N619AA AA 1990 Fixed wing mu~
## 4 2013 2013-01-01 05:00:00 JFK BQN N804JB B6 2012 Fixed wing mu~
## 5 2013 2013-01-01 06:00:00 LGA ATL N668DN DL 1991 Fixed wing mu~
## 6 2013 2013-01-01 05:00:00 EWR ORD N39463 UA 2012 Fixed wing mu~
```

```
## 7 2013 2013-01-01 06:00:00 EWR FLL N516JB B6 2000 Fixed wing mu-
## 8 2013 2013-01-01 06:00:00 LGA IAD N829AS EV 1998 Fixed wing mu-
## 9 2013 2013-01-01 06:00:00 JFK MCO N593JB B6 2004 Fixed wing mu-
## 10 2013 2013-01-01 06:00:00 LGA ORD N3ALAA AA NA <NA>
## # i 336,766 more rows
## # i 6 more variables: manufacturer <chr>, model <chr>, engines <int>,
## # seats <int>, speed <int>, engine <chr>
```

Both `year` columns are retained in the result.

The expression `join_by(tailnum)` in the code above is a shorthand for `join_by(tailnum == tailnum)`, and this type of join is also called an **equi join**.

Example of a non-equi-join:

```
flights2 |>
  left_join(airports, join_by(dest == faa))

## # A tibble: 336,776 x 13
##   year time_hour      origin dest tailnum carrier name      lat lon
##   <int> <dtm>      <chr> <chr> <chr> <chr> <chr>    <dbl> <dbl>
## 1 2013 2013-01-01 05:00:00 EWR IAH N14228 UA George Bu~ 30.0 -95.3
## 2 2013 2013-01-01 05:00:00 LGA IAH N24211 UA George Bu~ 30.0 -95.3
## 3 2013 2013-01-01 05:00:00 JFK MIA N619AA AA Miami Intl 25.8 -80.3
## 4 2013 2013-01-01 05:00:00 JFK BQN N804JB B6 <NA> NA NA
## 5 2013 2013-01-01 06:00:00 LGA ATL N668DN DL Hartsfiel~ 33.6 -84.4
## 6 2013 2013-01-01 05:00:00 EWR ORD N39463 UA Chicago O~ 42.0 -87.9
## 7 2013 2013-01-01 06:00:00 EWR FLL N516JB B6 Fort Laud~ 26.1 -80.2
## 8 2013 2013-01-01 06:00:00 LGA IAD N829AS EV Washingto~ 38.9 -77.5
## 9 2013 2013-01-01 06:00:00 JFK MCO N593JB B6 Orlando I~ 28.4 -81.3
## 10 2013 2013-01-01 06:00:00 LGA ORD N3ALAA AA Chicago O~ 42.0 -87.9
## # i 336,766 more rows
## # i 4 more variables: alt <dbl>, tz <dbl>, dst <chr>, tzone <chr>
```

```
flights2 |>
  left_join(airports, join_by(origin == faa))

## # A tibble: 336,776 x 13
##   year time_hour      origin dest tailnum carrier name      lat lon
##   <int> <dtm>      <chr> <chr> <chr> <chr> <chr>    <dbl> <dbl>
## 1 2013 2013-01-01 05:00:00 EWR IAH N14228 UA Newark Li~ 40.7 -74.2
## 2 2013 2013-01-01 05:00:00 LGA IAH N24211 UA La Guardia 40.8 -73.9
## 3 2013 2013-01-01 05:00:00 JFK MIA N619AA AA John F Ke~ 40.6 -73.8
## 4 2013 2013-01-01 05:00:00 JFK BQN N804JB B6 John F Ke~ 40.6 -73.8
## 5 2013 2013-01-01 06:00:00 LGA ATL N668DN DL La Guardia 40.8 -73.9
## 6 2013 2013-01-01 05:00:00 EWR ORD N39463 UA Newark Li~ 40.7 -74.2
## 7 2013 2013-01-01 06:00:00 EWR FLL N516JB B6 Newark Li~ 40.7 -74.2
## 8 2013 2013-01-01 06:00:00 LGA IAD N829AS EV La Guardia 40.8 -73.9
## 9 2013 2013-01-01 06:00:00 JFK MCO N593JB B6 John F Ke~ 40.6 -73.8
## 10 2013 2013-01-01 06:00:00 LGA ORD N3ALAA AA La Guardia 40.8 -73.9
## # i 336,766 more rows
## # i 4 more variables: alt <dbl>, tz <dbl>, dst <chr>, tzone <chr>
```

The other mutating joins like `inner_join()`, `right_join()`, and `full_join()` have the same interface as `left_join()` – they differ in which rows they keep:

- `left_join(x, y)` keeps all the rows in `x`.

- `right_join(x, y)` keeps all the rows in `y`.
- `full_join(x, y)` keeps all rows in either `x` or `y`.
- `inner_join(x, y)` only keeps rows that occur in both `x` and `y`.

Filtering joins

- main action of a **filtering join** is to filter the rows.
Common types of filtering joins:
- **semi-joins**: keep all rows in `x` that have a match in `y`.
- **anti-joins**: return all rows in `x` that don't have a match in `y`.

Semi-join examples:

```
airports |>
  semi_join(flights2, join_by(faa == origin))
```

```
## # A tibble: 3 x 8
```

	faa	name	lat	lon	alt	tz	dst	tzone
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<chr>
## 1	EWB	Newark Liberty Intl	40.7	-74.2	18	-5	A	America/New_York
## 2	JFK	John F Kennedy Intl	40.6	-73.8	13	-5	A	America/New_York
## 3	LGA	La Guardia	40.8	-73.9	22	-5	A	America/New_York

```
airports |>
  semi_join(flights2, join_by(faa == dest))
```

```
## # A tibble: 101 x 8
```

	faa	name	lat	lon	alt	tz	dst	tzone
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<chr>
## 1	ABQ	Albuquerque International Sunport	35.0	-107.	5355	-7	A	Ameri~
## 2	ACK	Nantucket Mem	41.3	-70.1	48	-5	A	Ameri~
## 3	ALB	Albany Intl	42.7	-73.8	285	-5	A	Ameri~
## 4	ANC	Ted Stevens Anchorage Intl	61.2	-150.	152	-9	A	Ameri~
## 5	ATL	Hartsfield Jackson Atlanta Intl	33.6	-84.4	1026	-5	A	Ameri~
## 6	AUS	Austin Bergstrom Intl	30.2	-97.7	542	-6	A	Ameri~
## 7	AVL	Asheville Regional Airport	35.4	-82.5	2165	-5	A	Ameri~
## 8	BDL	Bradley Intl	41.9	-72.7	173	-5	A	Ameri~
## 9	BGR	Bangor Intl	44.8	-68.8	192	-5	A	Ameri~
## 10	BHM	Birmingham Intl	33.6	-86.8	644	-6	A	Ameri~

```
## # i 91 more rows
```

Anti-join examples:

```
# Find rows that are missing from `airports` based on flights
# that don't have a matching destination airport
flights2 |>
  anti_join(airports, join_by(dest == faa)) |>
  distinct(dest)
```

```
## # A tibble: 4 x 1
```

	dest
	<chr>
## 1	BQN
## 2	SJU
## 3	STT
## 4	PSE

```
# Find which `tailnum`s are missing from `planes`
flights2 |>
  anti_join(planes, join_by(tailnum)) |>
  distinct(tailnum)
```

```
## # A tibble: 722 x 1
##   tailnum
##   <chr>
## 1 N3ALAA
## 2 N3DUAA
## 3 N542MQ
## 4 N730MQ
## 5 N9EAMQ
## 6 N532UA
## 7 N3EMAA
## 8 N518MQ
## 9 N3BAAA
## 10 N3CYAA
## # i 712 more rows
```

How do joins work?

Assuming that the first dataset is **x** and the second dataset is **y**:

- **left join** keeps all observations in **x**, and adds columns from **y** where the key matches.
- **right join** keeps all observations in **y**, and adds columns from **x** where the key matches.
- **full join** keeps all observations that appear in **x** or **y**. Every row of **x** and **y** is included in the output.
- **semi-join** keeps rows in **x** that have one or more matches in **y**.
- **anti-join** keeps rows in **x** that match zero rows in **y**.

Non-equi joins

Instead of matching only when the keys of **x** and **y** match (aka equi-join), non-equi joins match whenever the keys of **x** and **y** are not equal.

- **cross join** matches everything, generating the Cartesian product of rows (output will be `nrow(x)` by `nrow(y)`).
- **inequality join** use inequality symbols like `<`, `<=`, `>`, `>=` instead of `==`.
- **rolling join** are similar to inequality joins but only finds the closest match.
- **overlap join** are a special type of inequality join designed to work with ranges.

Examples:

Cross join

```
df <- tibble(name = c("John", "Simon", "Tracy", "Max"))
df |> cross_join(df)
```

```
## # A tibble: 16 x 2
##   name.x name.y
##   <chr> <chr>
## 1 John  John
## 2 John  Simon
## 3 John  Tracy
```

```
## 4 John Max
## 5 Simon John
## 6 Simon Simon
## 7 Simon Tracy
## 8 Simon Max
## 9 Tracy John
## 10 Tracy Simon
## 11 Tracy Tracy
## 12 Tracy Max
## 13 Max John
## 14 Max Simon
## 15 Max Tracy
## 16 Max Max
```

Example using inequality join, to generate all combinations of **x** and **y**:

```
df <- tibble(
  id = 1:4,
  name = c("John", "Simon", "Tracy", "Max")
)

df |> inner_join(df, join_by(id < id))
```

```
## # A tibble: 6 x 4
##   id.x name.x id.y name.y
##   <int> <chr> <int> <chr>
## 1     1 John     2 Simon
## 2     1 John     3 Tracy
## 3     1 John     4 Max
## 4     2 Simon    3 Tracy
## 5     2 Simon    4 Max
## 6     3 Tracy    4 Max
```

Example of a Rolling join, used on the following situation:

- a company having regular quarterly parties.
- parties are always held on a Monday.
- parties are not held when a lot of people are on holiday (party is pushed 1 week back).

```
# original party schedule
parties <- tibble(
  q = 1:4,
  party = ymd(c("2022-1-10", "2022-04-04", "2022-07-11", "2022-10-03"))
)

# table employee birthdays
set.seed(123)
employees <- tibble(
  name = sample(babynames::babynames$name, 100),
  birthday = ymd("2022-01-01") + (sample(365, 100, replace = TRUE) - 1)
)

parties
```

```
## # A tibble: 4 x 2
##       q party
##   <int> <date>
```

```
## 1      1 2022-01-10
## 2      2 2022-04-04
## 3      3 2022-07-11
## 4      4 2022-10-03
```

```
employees
```

```
## # A tibble: 100 x 2
##   name      birthday
##   <chr>     <date>
## 1 Kemba    2022-01-22
## 2 Orean    2022-06-26
## 3 Kirstyn  2022-02-11
## 4 Amparo   2022-11-11
## 5 Belen    2022-03-25
## 6 Rayshaun 2022-01-11
## 7 Brazil   2022-05-01
## 8 Chaston  2022-10-29
## 9 Reyn     2022-03-26
## 10 Ogechi  2022-12-31
## # i 90 more rows
```

- For each employee, we want to find the last party date that comes before, or on, their birthday.

```
employees |>
  left_join(parties, join_by(closest(birthday >= party)))
```

```
## # A tibble: 100 x 4
##   name      birthday    q party
##   <chr>     <date>   <int> <date>
## 1 Kemba    2022-01-22     1 2022-01-10
## 2 Orean    2022-06-26     2 2022-04-04
## 3 Kirstyn  2022-02-11     1 2022-01-10
## 4 Amparo   2022-11-11     4 2022-10-03
## 5 Belen    2022-03-25     1 2022-01-10
## 6 Rayshaun 2022-01-11     1 2022-01-10
## 7 Brazil   2022-05-01     2 2022-04-04
## 8 Chaston  2022-10-29     4 2022-10-03
## 9 Reyn     2022-03-26     1 2022-01-10
## 10 Ogechi  2022-12-31     4 2022-10-03
## # i 90 more rows
```

Example using overlap join

```
parties <- tibble(
  q = 1:4,
  party = ymd(c("2022-01-10", "2022-04-04", "2022-07-11", "2022-10-03")),
  start = ymd(c("2022-01-01", "2022-04-04", "2022-07-11", "2022-10-03")),
  end = ymd(c("2022-04-03", "2022-07-11", "2022-10-02", "2022-12-31"))
)
parties
```

```
## # A tibble: 4 x 4
##   q party      start      end
##   <int> <date>   <date>   <date>
## 1     1 2022-01-10 2022-01-01 2022-04-03
## 2     2 2022-04-04 2022-04-04 2022-07-11
```

```
## 3      3 2022-07-11 2022-07-11 2022-10-02
## 4      4 2022-10-03 2022-10-03 2022-12-31
```

- We want to check if the periods don't overlap. One way to approach this is by using a self-join to check if any start-end interval overlap with another:

```
parties |>
  inner_join(parties, join_by(overlaps(start, end, start, end), q < q)) |>
  select(start.x, end.x, start.y, end.y)
```

```
## # A tibble: 1 x 4
##   start.x   end.x   start.y   end.y
##   <date>   <date>   <date>   <date>
## 1 2022-04-04 2022-07-11 2022-07-11 2022-10-02
```

- Now we can fix the overlaps:

```
parties <- tibble(
  q = 1:4,
  party = ymd(c("2022-01-10", "2022-04-04", "2022-07-11", "2022-10-03")),
  start = ymd(c("2022-01-01", "2022-04-04", "2022-07-11", "2022-10-03")),
  end = ymd(c("2022-04-03", "2022-07-10", "2022-10-02", "2022-12-31"))
)
```

- Finally we can math each employee to the party. The `unmatched = "error"` argument is useful here because we want to know if any employees didn't get assigned to a party.

```
employees |>
  inner_join(parties, join_by(between(birthday, start, end)), unmatched = "error")
```

```
## # A tibble: 100 x 6
##   name      birthday      q party      start      end
##   <chr>    <date>    <int> <date>    <date>    <date>
## 1 Kemba    2022-01-22      1 2022-01-10 2022-01-01 2022-04-03
## 2 Orian    2022-06-26      2 2022-04-04 2022-04-04 2022-07-10
## 3 Kirstyn  2022-02-11      1 2022-01-10 2022-01-01 2022-04-03
## 4 Amparo   2022-11-11      4 2022-10-03 2022-10-03 2022-12-31
## 5 Belen    2022-03-25      1 2022-01-10 2022-01-01 2022-04-03
## 6 Rayshaun 2022-01-11      1 2022-01-10 2022-01-01 2022-04-03
## 7 Brazil   2022-05-01      2 2022-04-04 2022-04-04 2022-07-10
## 8 Chaston  2022-10-29      4 2022-10-03 2022-10-03 2022-12-31
## 9 Reyn     2022-03-26      1 2022-01-10 2022-01-01 2022-04-03
## 10 Ogechi  2022-12-31      4 2022-10-03 2022-10-03 2022-12-31
## # i 90 more rows
```